Can a 100M-Parameter Transformer Learn Simple Python Programming Tasks?

George Yuan
1012152224, zhijie.yuan@mail.utoronto.ca
GitHub: https://github.com/ShiinaMahiruOvO/aps360-small-code-lm

Introduction

Large code models can solve complex programming problems, but their scale makes it difficult to determine how much ability can be learned with a course-project compute budget. This project asks whether a decoder-only Transformer with approximately 100 million parameters, pretrained from scratch on Python code, can generate correct solutions to beginner-level programming tasks. These tasks include basic string and collection manipulation, linear scans and simple algorithms, and introductory data-structure operations such as binary search tree queries.

Our main architectural improvement is a *looped Transformer* that applies a shared group of layers more than once. This increases effective computational depth without proportionally increasing the number of parameters. We compare it with a standard Transformer trained on the same data and evaluate both models by running generated programs against hidden unit tests. Deep learning is appropriate because natural-language specifications and programs are variable-length sequences with many valid implementations, making fixed rules or templates too brittle. Our main question is whether iterative computation improves functional correctness, especially when a task combines several individually simple requirements.

This is an individual project. I will implement and document the data pipeline, model architecture, training procedures, execution-based evaluation, and final analysis. Configuration files, fixed random seeds, and version-controlled scripts will be used to keep the experiments reproducible.

Illustration

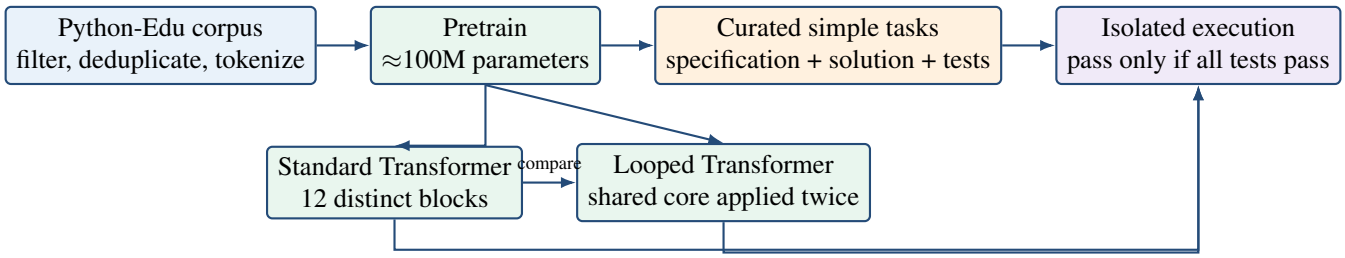


Figure 1: Project pipeline. The standard and looped Transformers use the same tokenizer, training data, fine-tuning tasks, and execution-based evaluation.

Background & Related Work

The Transformer established attention-only sequence modelling and forms the basis of our decoder [7]; RoPE supplies position information to its attention layers [6]. Codex and HumanEval demonstrated the importance of evaluating generated code through execution rather than textual similarity [3]. MBPP provides short Python problems close to our intended difficulty [1], while SmolLM-Corpus provides the educational Python source used for pretraining [2]. Most closely related to our architectural experiment, Universal Transformers repeatedly apply shared layers and report benefits on algorithmic tasks [4]. We apply this general idea to small-scale code generation and compare it directly with a conventional Transformer.

Data Processing

For pretraining, we use the Python-Edu subset of SmolLM-Corpus [2]. It contains Python files selected for educational quality. Our pipeline removes malformed, generated, extremely short, and duplicate files; checks that retained samples parse as Python; and separates repositories between training and validation to reduce leakage. We train an 8,192-token byte-pair tokenizer on the cleaned

text and convert the corpus into memory-mapped token shards. The processed corpus contains approximately 498 million training tokens and 55 million validation tokens.

For supervised fine-tuning, we combine easy problems filtered from KodCode-Light-RL-10K [5] with tasks generated by us. The task groups cover basic strings and collections, simple counting/searching/sorting algorithms, and introductory data structures. Our generated BST subset provides a controlled structured case study with prompts, reference solutions, and unit tests. We remove incomplete problems, invalid reference code, answer leakage, and duplicate statements or solutions. Training, validation, and evaluation sets are separated before prompt variants are produced. Evaluation uses a reviewed easy subset of MBPP and held-out generated tasks, with overlap removed from the fine-tuning data. Reference solutions must pass their tests before inclusion, and generated programs are executed in an isolated environment with time and resource limits.

Architecture

The baseline is a decoder-only Transformer with an 8,192-token vocabulary, context length 1,024, hidden width 768, 12 layers, 12 attention heads, RMSNorm, RoPE, and SwiGLU, totalling approximately 97.5M parameters. It is pretrained with next-token prediction and then supervised on instruction–solution pairs.

The looped model keeps two input blocks and two output blocks, while its eight middle blocks form a shared core that is executed twice. It therefore retains approximately the same number of distinct parameters as the baseline but has an effective depth of 20 layers. This provides a controlled test of whether a small model benefits more from iterative computation than from using each parameter only once. If time permits, we will test lightweight iteration-specific adapters and execution-based reinforcement learning as secondary extensions.

Baseline Model

The standard 12-layer Transformer is the primary baseline. It uses the same tokenizer, pretraining corpus, fine-tuning data, optimizer, and decoding settings as the looped model, making the architecture the main experimental variable. Functional correctness is measured with pass@1: a task passes only when the generated program satisfies every hidden test. We will report overall results, results by task group, parameter count, and training and inference cost. A small public code model may also be included as an external reference, but the parameter-matched standard Transformer remains the main comparison.

Ethical Considerations

Public code can contain licensing, attribution, privacy, and memorization risks, so we document dataset provenance and remove duplicates where possible. Generated programs may be incorrect or insecure; this model is an educational prototype, not a deployment-ready programming assistant. Untrusted completions are executed only in an isolated environment without network access. We will report failures and possible benchmark contamination and avoid claims beyond the beginner-level tasks studied here.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. Smollm-corpus. Hugging Face dataset, 2024. URL https://huggingface.co/datasets/HuggingFaceTB/smollm-corpus.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[5] KodCode. Kodcode-light-rl-10k. Hugging Face dataset, 2025. URL https://huggingface.co/datasets/KodCode/KodCode-Light-RL-10K.

[6] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.